

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

FACULTAD DE INGENIERIA EN SISTEMAS DE INFORMACION

SEDE: PORTALES PLAN: DOMINGO

CURSO: PROGRAMACION II SECCION: C



TEMA:

COLECCIONES, EXEPCIONES Y LAMBDA

HARVEY DANIEL RAMIREZ BURRION

CARNÉ: 9941-24-17982

FECHA: GUATEMALA, 27 DE SEPTIEMBRE DE 2025

REPOSITORIO 1: List_ArrayList

Descripción:

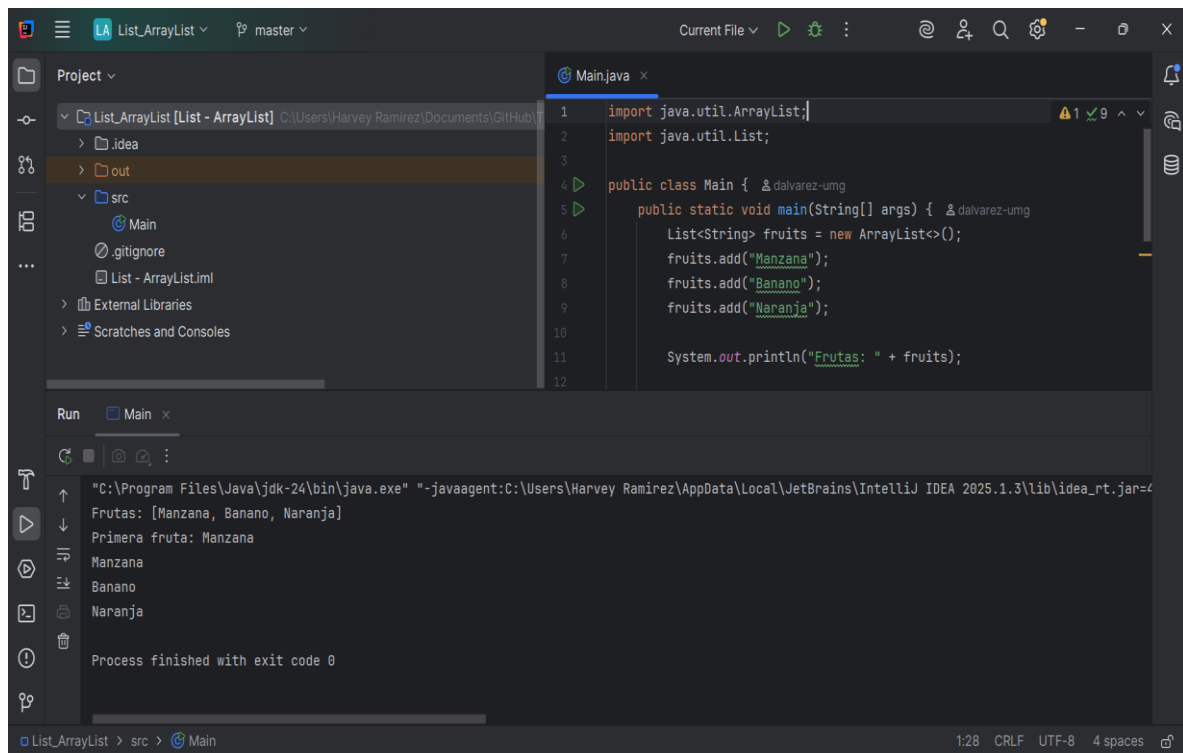
Este programa demuestra el uso básico de Listas en Java, específicamente con ArrayList, que es una de las estructuras de datos más utilizadas para almacenar colecciones de elementos.

Conceptos Aplicados

- **List Interface:** Interfaz que define operaciones para listas
- **ArrayList:** Implementación de lista usando arreglos dinámicos
- **Generics:** Uso de <String> para tipo seguro de datos
- **Iteración:** Recorrido de elementos con for-each

Qué hace el programa

1. Crea una lista de frutas usando ArrayList
2. Agrega 3 frutas diferentes a la lista (Solo que Manzana se repite)
3. Accede al primer elemento usando get(0)
4. Recorre e imprime todas las frutas usando un bucle for-each



The screenshot displays the IntelliJ IDEA IDE with a project named 'List_ArrayList'. The 'Main.java' file is open, showing the following code:

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Main {
5     public static void main(String[] args) {
6         List<String> fruits = new ArrayList<>();
7         fruits.add("Manzana");
8         fruits.add("Banano");
9         fruits.add("Naranja");
10
11         System.out.println("Frutas: " + fruits);
12     }
13 }
```

The 'Run' button is clicked, and the output console shows the following execution results:

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey_Ramirez\AppData\Local\JetBrains\IntelliJ IDEA 2025.1.3\lib\idea_rt.jar=4
Frutas: [Manzana, Banano, Naranja]
Primera fruta: Manzana
Manzana
Banano
Naranja
Process finished with exit code 0
```

The status bar at the bottom indicates the file encoding is UTF-8 and the line length is 128 characters.

REPOSITORIO 2: Set_HashSet

Descripción Breve

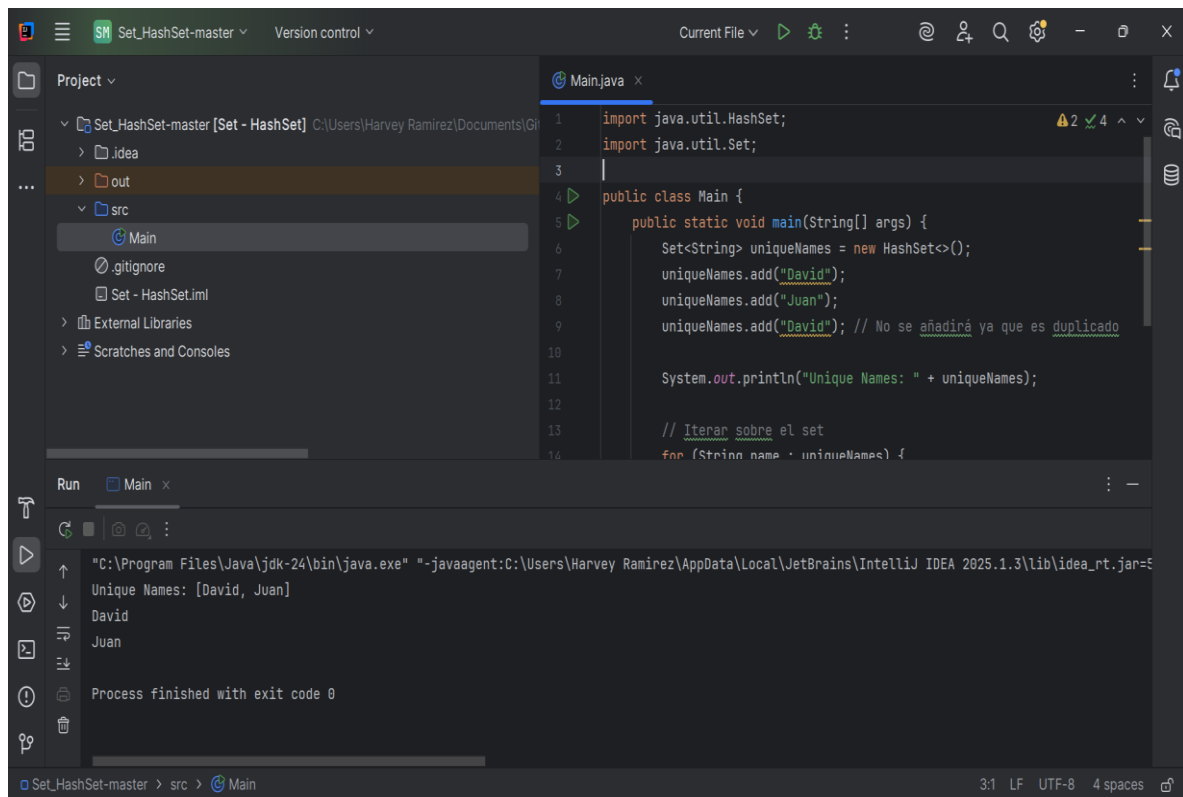
Este programa muestra el funcionamiento de Set en Java, específicamente con HashSet, que es una colección que no permite elementos duplicados y no mantiene un orden específico.

Conceptos Aplicados

- **Set Interface:** Interfaz para conjuntos sin elementos repetidos
- **HashSet:** Implementación usando tablas hash para almacenamiento
- **Unicidad:** Característica principal que elimina duplicados automáticamente

Qué hace el programa

1. Crea un HashSet de nombres
2. Intenta agregar 3 nombres, incluyendo "David" dos veces
3. Automáticamente elimina el duplicado
4. Itera e imprime los nombres únicos restantes



```
1 import java.util.HashSet;
2 import java.util.Set;
3
4 public class Main {
5     public static void main(String[] args) {
6         Set<String> uniqueNames = new HashSet<>();
7         uniqueNames.add("David");
8         uniqueNames.add("Juan");
9         uniqueNames.add("David"); // No se añadirá ya que es duplicado
10
11         System.out.println("Unique Names: " + uniqueNames);
12
13         // Iterar sobre el set
14         for (String name : uniqueNames) {
```

Run Main x

"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey Ramirez\AppData\Local\JetBrains\IntelliJ IDEA 2025.1.3\lib\idea_rt.jar=5" -jar C:\Users\Harvey Ramirez\IdeaProjects\Set_HashSet-master\out\Set_HashSet-master.jar

Unique Names: [David, Juan]

David

Juan

Process finished with exit code 0

REPOSITORIO 3: Map_HashMap

Descripción Breve

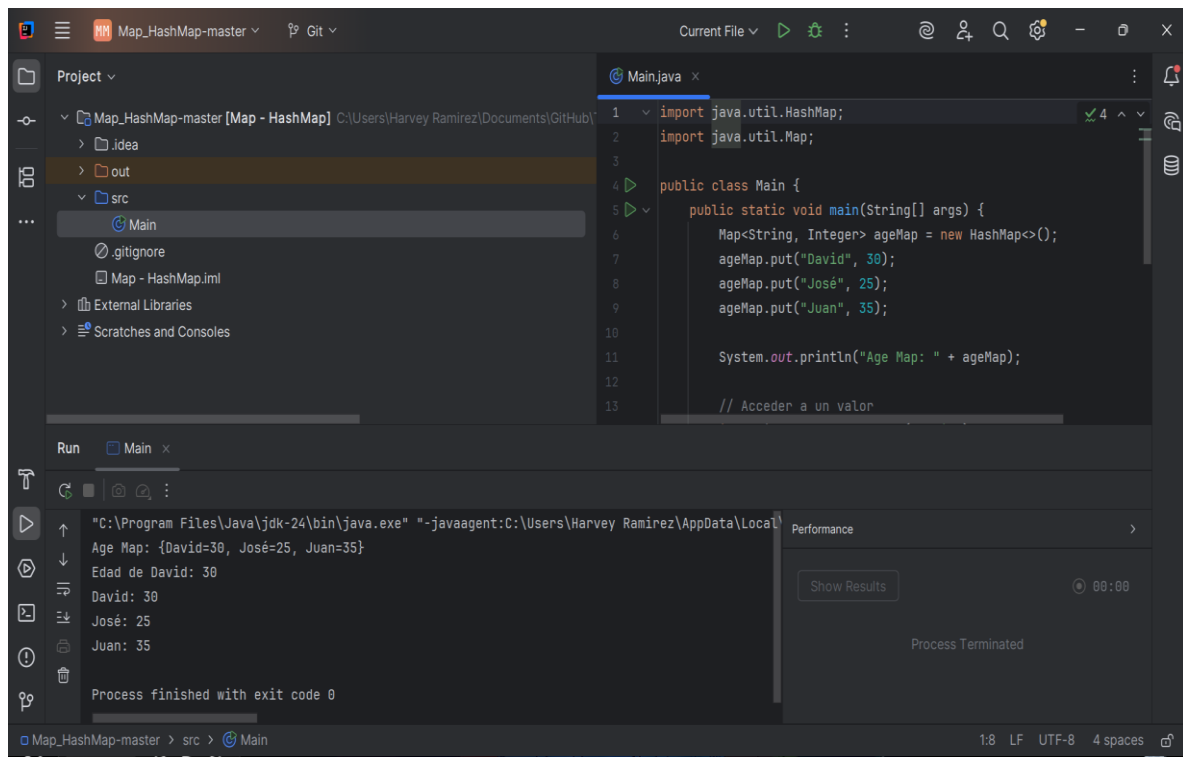
Este programa demuestra el uso de Map en Java, específicamente con HashMap, que es una estructura que almacena datos en pares clave-valor (como un diccionario).

Conceptos Aplicados

- **Map Interface:** Interfaz para almacenar pares clave-valor
- **HashMap:** Implementación usando tablas hash
- **Key-Value Pairs:** Cada elemento tiene una clave única y un valor asociado

Qué hace el programa

1. Crea un HashMap que asocia nombres (clave) con edades (valor)
2. Agrega 3 personas con sus respectivas edades
3. Accede a un valor específico usando su clave
4. Itera sobre todas las entradas mostrando nombre y edad



```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class Main {
5     public static void main(String[] args) {
6         Map<String, Integer> ageMap = new HashMap<>();
7         ageMap.put("David", 30);
8         ageMap.put("José", 25);
9         ageMap.put("Juan", 35);
10
11         System.out.println("Age Map: " + ageMap);
12
13         // Acceder a un valor
```

Run Main x

"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey Ramirez\AppData\Local\..."

Age Map: {David=30, José=25, Juan=35}

Edad de David: 30

David: 30

José: 25

Juan: 35

Process finished with exit code 0

REPOSITORIO 4: Generic_Class

Descripción Breve

Este programa demuestra el uso de Clases Genéricas en Java, que permiten crear clases que pueden trabajar con diferentes tipos de datos sin necesidad de repetir código.

Conceptos Aplicados

- **Generics:** Parámetros de tipo para crear clases flexibles
- **Type Safety:** Seguridad de tipos en tiempo de compilación
- **Reutilización:** Una misma clase funciona para múltiples tipos de datos

Qué hace el programa

1. Crea una clase genérica `Box<T>` que puede contener cualquier tipo de objeto
2. Instancia la caja para `String` y almacena un mensaje
3. Instancia la caja para `Integer` y almacena un número
4. Recupera y muestra el contenido de ambas cajas

```
1 public class Box<T> {
2     private T content; 2 usages
3
4     > public void setContent(T content) { this.content = content; }
7
8     > public T getContent() { return content; }
11
12 > public static void main(String[] args) {
13     Box<String> stringBox = new Box<>();
14     stringBox.setContent("Hola a todos");
15     System.out.println("Contenido de la caja: " + stringBox.getContent());
16
17     Box<Integer> integerBox = new Box<>();
18     integerBox.setContent(123);
19     System.out.println("Contenido de la caja: " + integerBox.getContent());
20 }
21 }
```

Run Console Output:

```
Contenido de la caja: Hola a todos
Contenido de la caja: 123
Process finished with exit code 0
```

REPOSITORIO 5: Generic_Method

Descripción Breve

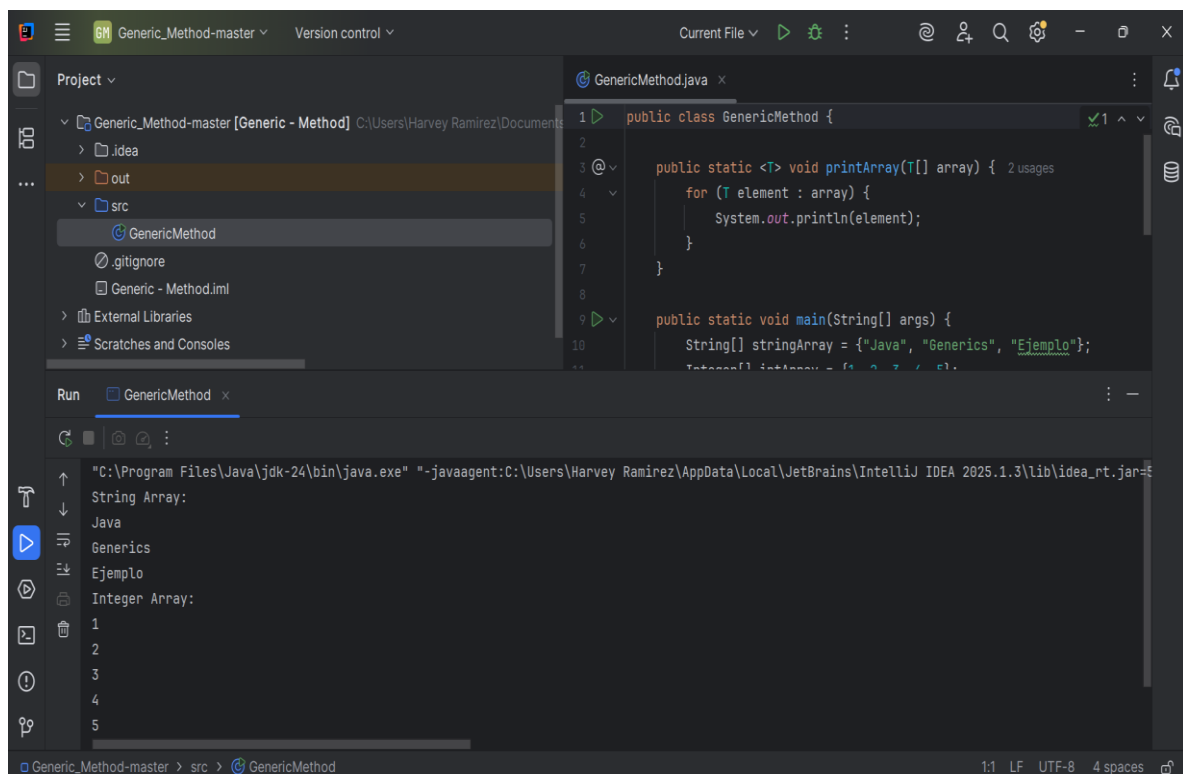
Este programa demuestra el uso de Métodos Genéricos en Java, que permiten crear métodos que pueden trabajar con diferentes tipos de datos sin necesidad de sobrecargarlos.

Conceptos Aplicados

- **Métodos Genéricos:** Métodos con parámetros de tipo
- **Flexibilidad:** Un mismo método funciona para múltiples tipos de arrays
- **Type Inference:** Java deduce automáticamente el tipo

Qué hace el programa

1. Crea un método genérico `printArray<T>` que imprime cualquier tipo de array
2. Usa el método con un array de Strings
3. Usa el mismo método con un array de Integers
4. Demuestra que un solo método sirve para diferentes tipos



REPOSITORIO 6: Try_Catch

Descripción Breve

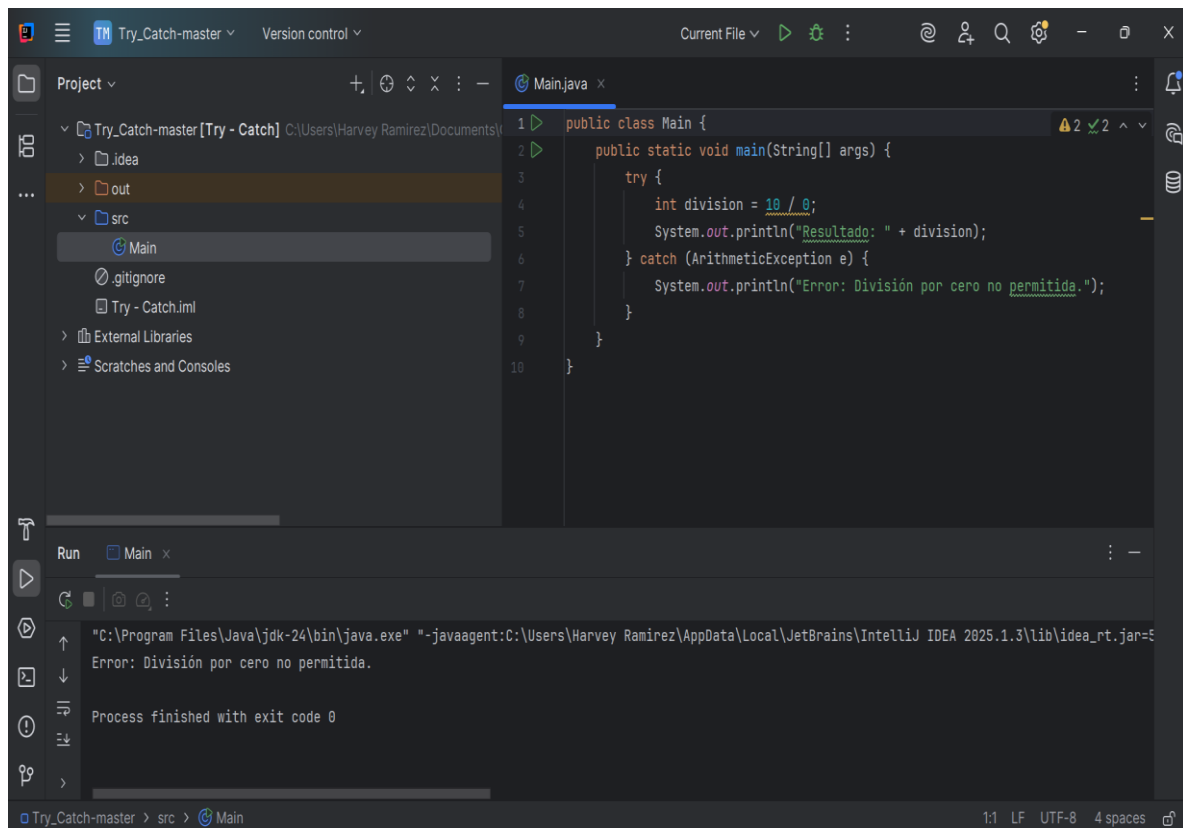
Este programa demuestra el manejo básico de excepciones en Java usando try-catch, que permite controlar errores durante la ejecución del programa sin que se detenga abruptamente.

Conceptos Aplicados

- **Try-Catch:** Bloque para capturar y manejar excepciones
- **ArithmeticException:** Excepción por operaciones matemáticas inválidas
- **Manejo de errores:** Control graceful de fallos en el programa

Qué hace el programa

1. Intenta realizar una división por cero (operación inválida)
2. Captura la excepción ArithmeticException que se genera
3. Muestra un mensaje de error controlado en lugar de que el programa se cierre



The screenshot shows an IDE with a project named 'Try_Catch-master'. The 'src' directory contains a file named 'Main.java'. The code in 'Main.java' is as follows:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         try {  
4             int division = 10 / 0;  
5             System.out.println("Resultado: " + division);  
6         } catch (ArithmeticException e) {  
7             System.out.println("Error: División por cero no permitida.");  
8         }  
9     }  
10 }
```

The 'Run' button is clicked, and the output console shows the following message:

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey Ramirez\AppData\Local\JetBrains\IntelliJ IDEA 2025.1.3\lib\idea_rt.jar=5  
Error: División por cero no permitida.  
Process finished with exit code 0
```

REPOSITORIO 7: Bloque_Finally

Descripción Breve

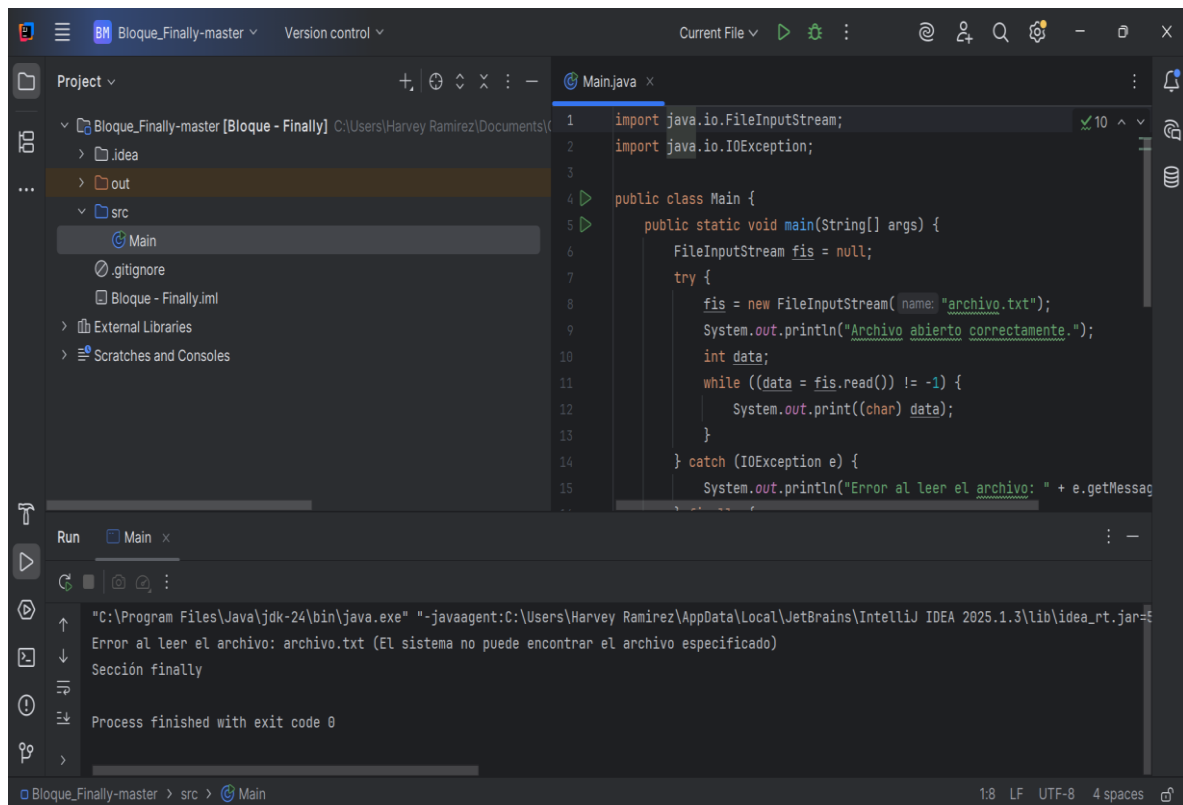
Este programa demuestra el uso del bloque finally en Java, que se ejecuta siempre sin importar si el código tuvo éxito o falló, ideal para liberar recursos como archivos, conexiones de base de datos, etc.

Conceptos Aplicados

- **Finally:** Bloque que siempre se ejecuta (con o sin excepciones)
- **Manejo de recursos:** Apertura y cierre seguro de archivos
- **Try-Catch-Finally:** Estructura completa de manejo de excepciones

Qué hace el programa

1. Intenta abrir y leer un archivo "archivo.txt"
2. Maneja posibles errores de lectura con catch
3. En el bloque finally asegura que el archivo se cierre correctamente
4. Garantiza que el recurso se libere, aunque falle la lectura



The screenshot shows an IDE with a project named 'Bloque_Finally-master'. The main file, 'Main.java', contains the following code:

```
1 import java.io.FileInputStream;
2 import java.io.IOException;
3
4 public class Main {
5     public static void main(String[] args) {
6         FileInputStream fis = null;
7         try {
8             fis = new FileInputStream("archivo.txt");
9             System.out.println("Archivo abierto correctamente.");
10            int data;
11            while ((data = fis.read()) != -1) {
12                System.out.print((char) data);
13            }
14        } catch (IOException e) {
15            System.out.println("Error al leer el archivo: " + e.getMessage());
16        } finally {
17            if (fis != null) {
18                try {
19                    fis.close();
20                } catch (IOException e) {
21                    System.out.println("Error al cerrar el archivo: " + e.getMessage());
22                }
23            }
24        }
25    }
26 }
```

The Run console shows the output of the program:

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey Ramirez\AppData\Local\JetBrains\IntelliJ IDEA 2025.1.3\lib\idea_rt.jar=5000:C:\Program Files\Java\jdk-24\bin" -Dfile.encoding=UTF-8
Error al leer el archivo: archivo.txt (El sistema no puede encontrar el archivo especificado)
Sección finally
Process finished with exit code 0
```


REPOSITORIO 8: Throw_Throws

Descripción Breve

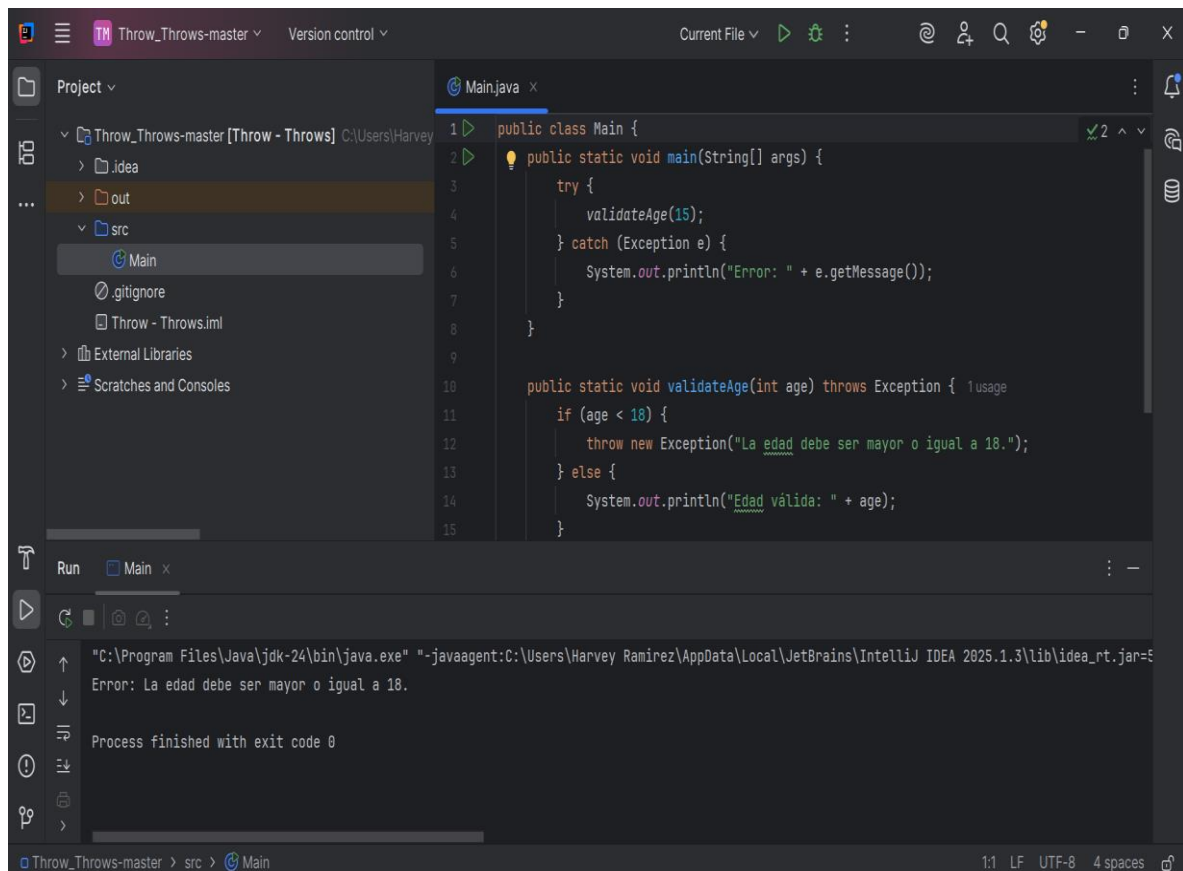
Este programa demuestra la diferencia entre `throw` y `throws` en Java, que permiten lanzar y declarar excepciones respectivamente, dando control sobre el manejo de errores personalizados.

Conceptos Aplicados

- **throw:** Palabra clave para lanzar una excepción manualmente
- **throws:** Declaración en método indicando que puede lanzar excepciones
- **Excepciones personalizadas:** Crear condiciones de error específicas

Qué hace el programa

1. Valida una edad (15 años) usando el método `validateAge`
2. El método lanza una excepción porque la edad es menor de 18
3. Captura y muestra el mensaje de error personalizado



The screenshot shows an IDE window with a project named "Throw_Throws-master". The left sidebar displays the project structure, including a "src" folder containing a "Main" class. The main editor area shows the code for "Main.java". The code defines a `Main` class with a `main` method that calls `validateAge(15)` and a `validateAge` method that throws an `Exception` if the age is less than 18. The `main` method catches the exception and prints an error message. The bottom panel shows the output of the program, which is "Error: La edad debe ser mayor o igual a 18." and "Process finished with exit code 0".

```
1 public class Main {
2     public static void main(String[] args) {
3         try {
4             validateAge(15);
5         } catch (Exception e) {
6             System.out.println("Error: " + e.getMessage());
7         }
8     }
9
10    public static void validateAge(int age) throws Exception { 1 usage
11        if (age < 18) {
12            throw new Exception("La edad debe ser mayor o igual a 18.");
13        } else {
14            System.out.println("Edad válida: " + age);
15        }
16    }
```

Run Main x

"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey Ramirez\AppData\Local\JetBrains\IntelliJ IDEA 2025.1.3\lib\idea_rt.jar=5" Error: La edad debe ser mayor o igual a 18.

Process finished with exit code 0

Throw_Throws-master > src > Main 1:1 LF UTF-8 4 spaces

REPOSITORIO 9: Excepciones_Personalizadas

Descripción Breve

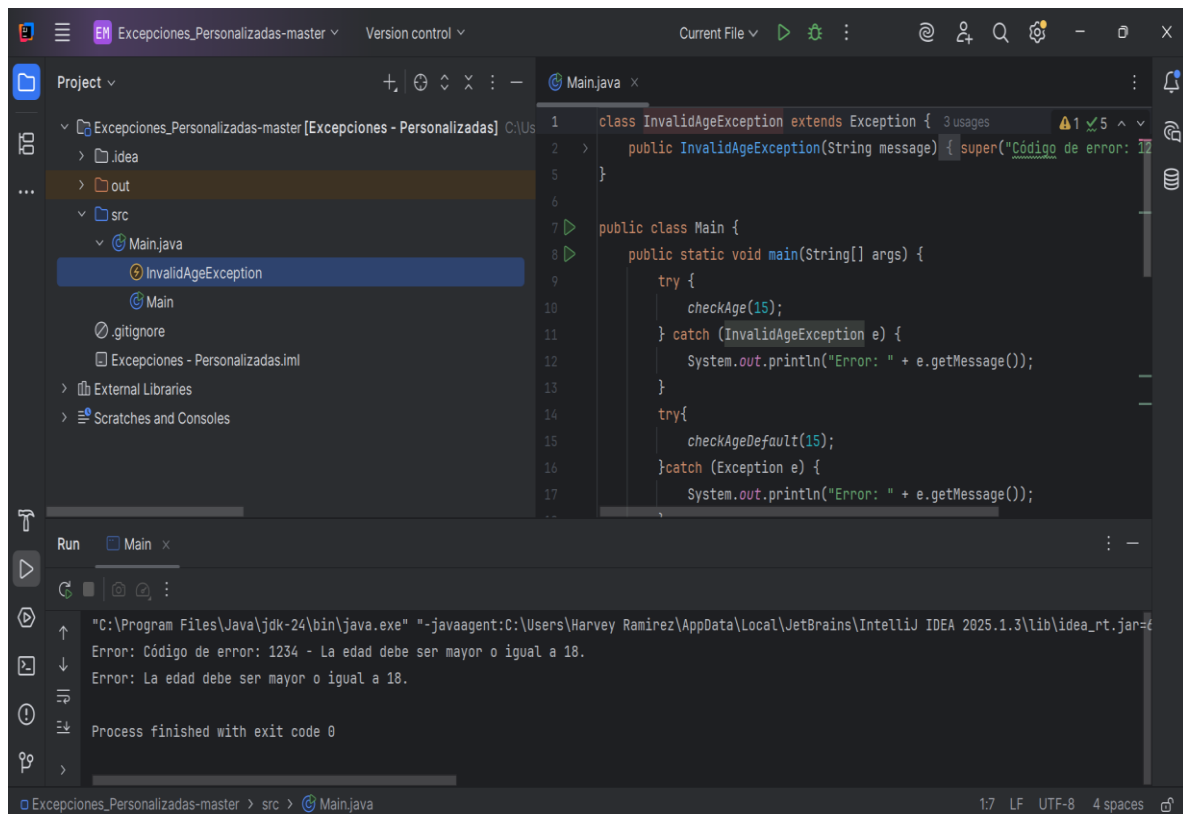
Este programa demuestra cómo crear y usar Excepciones Personalizadas en Java, permitiendo definir errores específicos para tu aplicación con mensajes y comportamientos personalizados.

Conceptos Aplicados

- **Excepciones personalizadas:** Crear tus propias clases de excepción
- **Herencia de Exception:** Extender la clase base Exception
- **Mensajes personalizados:** Información específica de error
- **Códigos de error:** Identificadores únicos para cada tipo de error

Qué hace el programa

1. Crea una excepción personalizada InvalidAgeException
2. Compara dos enfoques: excepción personalizada vs excepción genérica
3. Demuestra las ventajas de usar excepciones específicas



```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) { super("Código de error: 1234 - La edad debe ser mayor o igual a 18."); }
}

public class Main {
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (InvalidAgeException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            checkAgeDefault(15);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Run Main

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Harvey Ramirez\AppData\Local\JetBrains\IntelliJ IDEA 2025.1.3\lib\idea_rt.jar=..."
Error: Código de error: 1234 - La edad debe ser mayor o igual a 18.
Error: La edad debe ser mayor o igual a 18.
Process finished with exit code 0
```